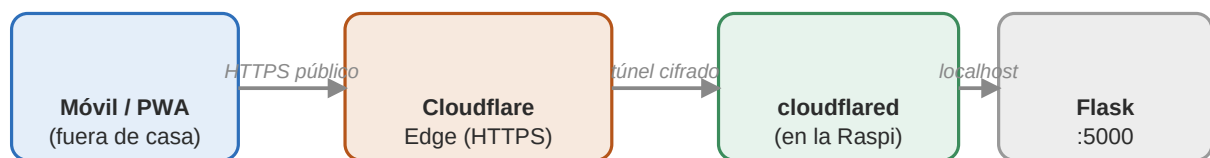


Dreame! / StockHogar

Publicar la app en internet con Cloudflare Tunnel

Guía paso a paso para exponer StockHogar (Flask, puerto 5000) desde una Raspberry Pi 3 a través de HTTPS público, sin abrir puertos en el router.



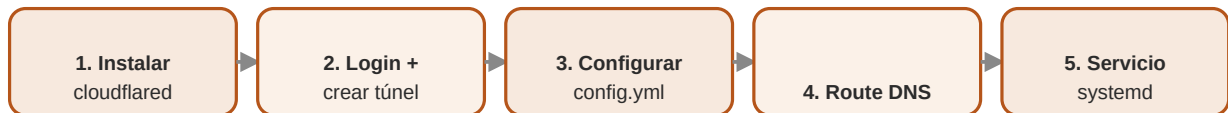
No se abre ningún puerto en el router. El túnel lo inicia siempre la Raspi hacia fuera.

Por qué esta opción: Cloudflare Tunnel crea una conexión saliente desde la Raspi hacia la red de Cloudflare. No hace falta abrir el puerto 5000 en el router, no dependes de tener IP pública fija, y Cloudflare gestiona el certificado HTTPS automáticamente. Esto es imprescindible para que la PWA sea instalable en iOS/Android y para que funcionen Service Worker, cámara y notificaciones push.

Requisitos previos

- Raspberry Pi 3 con StockHogar ya instalado y funcionando en el puerto 5000 (ver docs/INSTALACION_RASPBERRY_PI.md).
- Una cuenta gratuita en Cloudflare (cloudflare.com).
- Un dominio propio gestionado en Cloudflare (ej. tudominio.com). No vale un dominio gratuito tipo DuckDNS: el dominio debe tener sus DNS delegados a Cloudflare.
- Acceso SSH a la Raspberry Pi.

Resumen de los 5 pasos



Paso 1. Instalar cloudflared en la Raspberry Pi

Conéctate por SSH a la Raspberry Pi y descarga el paquete oficial para su arquitectura (ARM). La Raspberry Pi 3 suele ser **armhf (armv7l)** con Raspbian de 32 bits, o **arm64** si usas un sistema operativo de 64 bits. Comprueba la arquitectura antes de descargar:

```
uname -m
# armv7l -> 32 bits (usa el paquete armhf)
# aarch64 -> 64 bits (usa el paquete arm64)
```

Descarga e instala el paquete .deb correspondiente:

```
# Para armv7l (Raspbian 32 bits):
curl -L --output cloudflared.deb \
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-armhf.deb

# Para aarch64 (64 bits):
curl -L --output cloudflared.deb \
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm64.deb

sudo dpkg -i cloudflared.deb
cloudflared --version
```

Si el último comando muestra una versión, la instalación fue correcta.

Paso 2. Autenticarse y crear el túnel

Este paso abre un enlace que debes abrir en el navegador de **cualquier** ordenador (no hace falta que sea en la propia Raspi) para autorizar el acceso a tu cuenta de Cloudflare:

```
cloudflared tunnel login
```

Se guardará un certificado en `~/.cloudflared/cert.pem`. A continuación, crea el túnel con un nombre identificativo:

```
cloudflared tunnel create stockhogar
```

El comando genera un identificador único (UUID) y un fichero de credenciales en `~/.cloudflared/<UUID>.json`. Anota el UUID, lo necesitarás en el paso siguiente.

Paso 3. Configurar el túnel (config.yml)

Crea el fichero de configuración que indica a Cloudflare a qué dirección local debe reenviar el tráfico público. Debe apuntar al mismo puerto que usa StockHogar (variable `STOCKHOGAR_PORT` del `.env`, por defecto 5000):

```
nano ~/.cloudflared/config.yml

tunnel: stockhogar
credentials-file: /home/pi/.cloudflared/<UUID>.json

ingress:
- hostname: app.tudominio.com
  service: http://localhost:5000
- service: http_status:404
```

Sustituye <UUID> por el generado en el paso 2 y `app.tudominio.com` por el subdominio que quieras usar. La última línea (`http_status:404`) es obligatoria: captura cualquier petición que no coincida con ningún hostname configurado.

Paso 4. Crear el registro DNS

Este comando crea automáticamente en Cloudflare el registro DNS tipo CNAME que enlaza tu subdominio con el túnel, sin tener que tocar nada manualmente en el panel:

```
cloudflared tunnel route dns stockhogar app.tudominio.com
```

A partir de aquí, cualquier petición a `https://app.tudominio.com` viaja cifrada por la red de Cloudflare hasta el túnel abierto por tu Raspi, sin pasar por el router.

Paso 5. Dejarlo funcionando como servicio (systemd)

Para que el túnel se reconecte solo si se reinicia la Raspberry Pi o se cae la conexión, instálalo como servicio del sistema en vez de dejarlo corriendo en una terminal abierta:

```
sudo cloudflared service install
sudo systemctl enable cloudflared
sudo systemctl start cloudflared
```

```
# Comprobar estado:
sudo systemctl status cloudflared
```

```
# Ver logs en vivo:
sudo journalctl -u cloudflared -f
```

Con esto, la Raspi mantiene el túnel abierto de forma permanente y arranca automáticamente al reiniciar.

Verificación

Comprobación	Cómo hacerlo	Resultado esperado
El túnel está activo	<code>sudo systemctl status cloudflared</code>	active (running)
El dominio resuelve	Abrir <code>https://app.tudominio.com</code> en el navegador del móvil	Carga la app StockHogar con candado HTTPS
Service Worker se registra	DevTools remoto o consola del navegador (<code>navigator.serviceWorker.getRegistrations()</code>)	Un registro con scope <code>"/</code>
Instalación en iOS	Safari > Compartir > "Añadir a pantalla de inicio"	Icono propio, abre en modo standalone
Instalación en Android	Chrome > menú > "Instalar aplicación"	Aparece banner de instalación automático

Notas y problemas frecuentes

- **Error 502 / Bad Gateway:** comprueba que el contenedor Flask está arrancado (`docker ps`) y que el puerto en `config.yml` coincide con `STOCKHOGAR_PORT` del `.env`.
- **El túnel no arranca tras reiniciar la Raspi:** revisa `sudo systemctl status cloudflared` y los logs con `journalctl -u cloudflared`; suele ser una ruta incorrecta al fichero de credenciales en `config.yml`.
- **Quieres varios servicios (ej. StockHogar + otra app):** añade más bloques bajo `ingress:` en `config.yml`, uno por hostname, antes de la línea final `http_status:404`.
- **Alternativa sin dominio propio:** si solo quieres acceso para ti mismo (no compartir la app con nadie más), Tailscale es más simple: crea una red privada tipo VPN sin necesidad de DNS público ni certificado, pero solo funciona en los dispositivos dados de alta en tu red privada.